

ASTBuilder Component

Langflow custom component — generates AST constraints and Z3 structures from formalized requirements.

```
from lfx.custom.custom_component.component import Component
from lfx.io import MessageTextInput, Output
from lfx.schema.message import Message
```

```
import json
import re
```

```
class ASTBuilder(Component):

    display_name = "AST Builder"
    description = (
        "Generates AST constraints and Z3 structures "
        "from formalized requirements."
    )
    icon = "code"
    name = "ASTBuilder"

    inputs = [
        MessageTextInput(
            name="input_value",
            display_name="Formalized JSON"
        )
    ]

    outputs = [
        Output(
            name="output",
            display_name="Formalized AST",
            method="build_output"
        )
    ]

    def normalize(self, text):

        if not text:
            return None

        text = re.sub(
            r"^[A-Za-z0-9_ ]",
            "",
            str(text)
        )

        text = (
            text.replace(" ", "_")
            .replace("-", "_")
        )

        text = re.sub(
            r"_{+}",
            "_",

```

```

        text
    )

    return text.strip("_")

def parse_input(self):
    text = str(self.input_value).strip()

    if text.startswith("```json"):
        text = text[7:]

    elif text.startswith("```"):
        text = text[3:]

    if text.endswith("```"):
        text = text[:-3]

    return json.loads(text.strip())

def build_requirement(self, req):
    action = req.get("action")
    condition = req.get("condition")

    if not action:
        req["ast_builder_status"] = (
            "MISSING_ACTION"
        )

        return req

    action_predicate = self.normalize(
        action
    )

    predicates = [
        {
            "name": action_predicate,
            "arity": 0,
            "source": action
        }
    ]

    variables = [
        {
            "name": action_predicate,
            "source": action
        }
    ]

    if condition:
        condition_predicate = (
            self.normalize(condition)
        )

```

```

    predicates.append(
        {
            "name": condition_predicate,
            "arity": 0,
            "source": condition
        }
    )

    variables.append(
        {
            "name": condition_predicate,
            "source": condition
        }
    )

    ast_constraints = [
        {
            "type": "IMPLIES",
            "antecedent": {
                "type": "VAR",
                "name":
                    condition_predicate
            },
            "consequent": {
                "type": "VAR",
                "name":
                    action_predicate
            }
        }
    ]

    fol_formula = (
        f"{condition_predicate}"
        f" -> "
        f"{action_predicate}"
    )

else:

    ast_constraints = [
        {
            "type": "VAR",
            "name": action_predicate
        }
    ]

    fol_formula = (
        action_predicate
    )

req["z3_signature"] = {
    "sorts": [],
    "predicates": predicates,
    "functions": [],
    "constants": []
}

```

```

req["variables"] = variables

req["fol_formula"] = (
    fol_formula
)

req["ast_constraints"] = (
    ast_constraints
)

req["constraints"] = [
    {
        "constraint_id": "C1",
        "ast_ref": 0,
        "source":
            req.get(
                "source_requirement",
                action
            )
    }
]

req["formalization_metadata"] = {
    "translation_confidence": 100,
    "formalization_type":
        "PROPOSITIONAL",
    "variable_count":
        len(variables),
    "constraint_count":
        len(ast_constraints)
}

req["consistency_result"] = (
    "READY_FOR_SOLVING"
)

req["consistency_explanation"] = ""

req["ast_builder_status"] = (
    "SUCCESS"
)

return req

def build_output(self) -> Message:

    try:

        data = self.parse_input()

        requirements = data.get(
            "requirements",
            []
        )

        if not requirements:

```

```

    result = {
        "requirements": [],
        "formalization_summary": {
            "detected_requirement_count": 0,
            "processed_count": 0,
            "formalizable_count": 0,
            "not_formalizable_count": 0,
            "formalization_performed": False
        }
    }

    return Message(
        text=json.dumps(
            {
                "DEBUG_AST_OUTPUT": result
            },
            indent=2,
            ensure_ascii=False
        )
    )

    processed = []

    success_count = 0

    for req in requirements:

        req = self.build_requirement(
            req
        )

        if (
            req.get(
                "ast_builder_status"
            )
            == "SUCCESS"
        ):
            success_count += 1

        processed.append(req)

    result = {
        "requirements":
            processed,

        "formalization_summary": {
            "detected_requirement_count":
                len(requirements),

            "processed_count":
                len(requirements),

            "formalizable_count":
                success_count,

            "not_formalizable_count":

```

```

        len(requirements)
        - success_count,

        "formalization_performed":
            True
    }
}

self.status = result

return Message(
    text=json.dumps(
        result,
        ensure_ascii=False,
        indent=2
    )
)

except Exception as ex:

    error_result = {
        "status": "ERROR",
        "message": str(ex)
    }

    return Message(
        text=json.dumps(
            error_result,
            ensure_ascii=False,
            indent=2
        )
    )

```